

Creating and Deploying Production-Ready Agentic Systems

Dimitris Tsirmpas

Athens University of Economics and Business

Archimedes, Athena Research Center

August 28, 2026

Part I

Autonomous LLM Agents

Outline

1 LLMs as System Components

- Inherent LLM limitations
- When to use LLMs?

2 Tool-Based Reasoning using ReAct

- Thought, Action, Observation
- Standardization

3 Agent-to-Agent Interaction

- Single-Agent vs. Multi-Agent
- Orchestrating Interaction
- What do we actually need?

Section 1

LLMs as System Components

Subsection 1

Inherent LLM limitations

Mathematics

LLMs are terrible at math

- Predict the most likely sequence of tokens, not the logically necessary derivation.
- Probabilistic nature → prone to hallucinations.
- Tokenization absolutely destroys numbers.

While OK for small numbers (they can memorize some patterns), they generally fail at larger ones.

Word counting

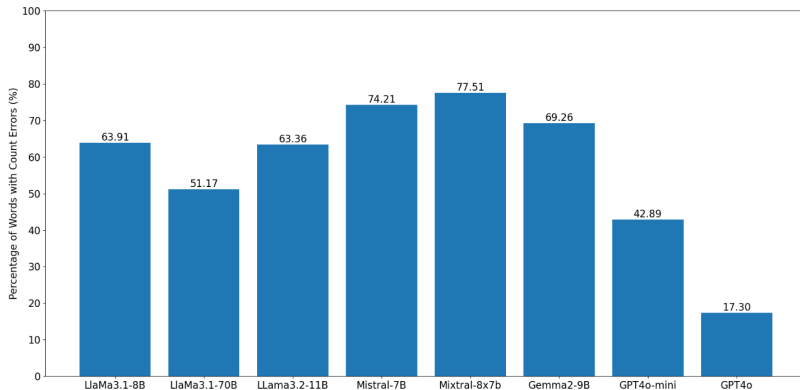


Figure: Percentage of words with errors on when counting letters for the different models
<https://arxiv.org/html/2412.18626v1>.

Character counting

Example

```
model.prompt("How many 'r's are in 'carr'?")  
>>> "There is one 'r' in 'carr'".
```

Indicative explanation

- The model can not find the word “carr” in its knowledge base.
- But it knows the word “car” and the fact that “carr” and “car” are highly correlated.
- “car” has one “r”, therefore this is the answer.

Deductive reasoning

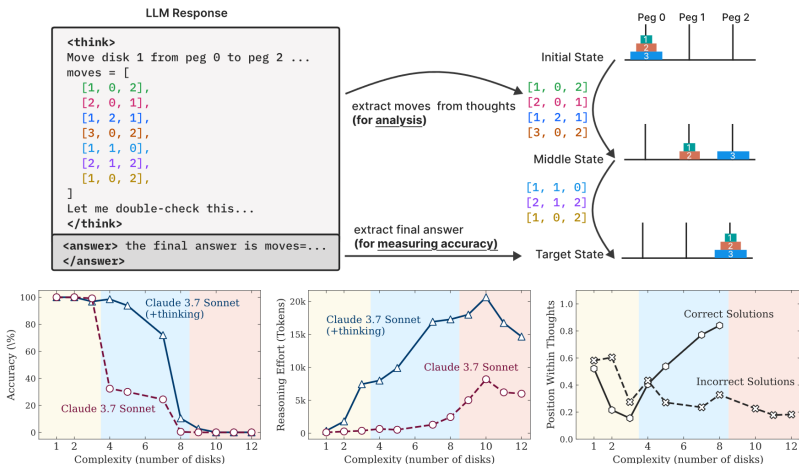


Figure: Percentage of words with errors on when counting letters for the different models

<https://arxiv.org/html/2412.18626v1>

General Limitations

LLMs cannot:

- Think.
- Behave.
- Deduct.

Why?

- LLMs are generally **excellent stochastic parrots**. Their **next-word-prediction** nature prevents them from understanding the world and its mechanisms.
- Finetuning, reasoning and in-context learning alleviate, but do not bypass this restriction.

A Note on Benchmarks

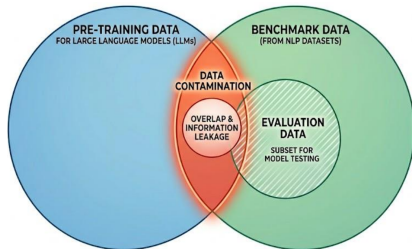
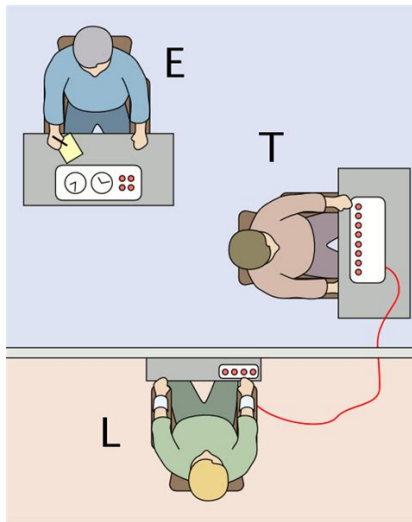


Figure: Credit: Electra Maria Papadopoulos.

Data Contamination

- Evaluation data appears in training.
- Web-scraping may include datasets used as training, validation, or benchmarks, unknowingly inflating reported performance.
- Sometimes on purpose (“benchmaxxing”).

A Note on Benchmarks



Behavior replication

- LLMs replicate the Milgram study.
- Emergent social property!
- ...or is it that the LLMs already know the results of the study?
- Perhaps, they implicitly follow what they know humans do.

A Note on Benchmarks

Definition (Benchmaxxing)

Optimizing a LLM's performance primarily for high scores on public **leaderboards** and **benchmarks**, rather than practical, and reliable real-world capability.

How?

- Training Data Contamination.
- Format Overfitting.
- Checkpoint Cherry-Picking.
- Prompt Engineering.

Section 1

LLMs as System Components

Subsection 2

When to use LLMs?

LLMs are not just task-specific tools

- So far, we have used LLMs as specialized tools for specific tasks.
- While useful, these tasks can generally be accomplished by **simpler** and more **scalable** models (with some performance loss).
- Where LLMs differ, is that they are **autonomous**, which allows for **previously impossible** use-cases.

When you are holding a hammer...

Suppose you are given the following task, with appropriate resources from management.

Find all reviews that concern competitors of our product, and tell us what percentage of them mention it.

When you are holding a hammer...

Our problem could be decomposed in three parts

- 1 Discovery: Find pages containing reviews.
- 2 Filtering: Find the HTML sections containing the reviews.
- 3 Analysis: Find which reviews reference the product.

Which of these must necessarily be solved with LLMs?
Which can be solved with traditional ways? What
trade-offs are we making in the latter case?

... you treat everything as if it were a nail

The easy solution

We instruct an LLM to retrieve reviews via RAG. We isolate the reviews using prompting, then instruct the LLM to find whether our product is mentioned.

Result

Code is deployed **within the day**. Management is billed **50,000€** in a week.

But maybe we can do better

Example: Scraping

- We use a traditional scrapper to find relevant reviews. For each page, we instruct the LLM to extract the reviews. We then run `reviews.str.contains(product_name).sum()`.
- **Result:** Code is deployed within **two days**. Management is billed **20€** in a week.

Summing up

LLMs as system components

- Ordinary tasks that are trivial to humans or through programming may be exceptionally hard for LLMs.
- This fact motivates us to combine them with standard tools instead of relying on them for everything.
- Generally, LLMs should be thought of as functions which can adapt to the input.

Outline

1 LLMs as System Components

- Inherent LLM limitations
- When to use LLMs?

2 Tool-Based Reasoning using ReAct

- Thought, Action, Observation
- Standardization

3 Agent-to-Agent Interaction

- Single-Agent vs. Multi-Agent
- Orchestrating Interaction
- What do we actually need?

Section 2

Tool-Based Reasoning using ReAct

Subsection 1

Thought, Action, Observation

Tool-Calling

Tool example

```
from smolagents import tool

@tool
def calculator(expr: str) -> float:
    """
    Evaluate a basic arithmetic expression.
    Args:
    expr: The math expression to evaluate, such as
    "2 + 3 * (4 / 2)".
    """
    return eval(expr, {"__builtins__": None}, {})
```

What is ReAct?

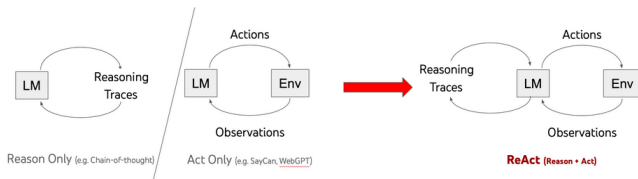


Figure: <https://arxiv.org/abs/2210.03629>

Definition

ReAct: A technique that interleaves reasoning steps (“Thought”) with Actions and Observations to guide an LLM through planning and tool use.

Tool calling example

```
{
  "tool_calls": [
    {
      "id": "calc1",
      "function": {
        "name": "calculate",
        "arguments": {
          "command": "calculator(
            '2 + 3 * (4 / 2)'"
        )
      }
    }
  ]
}
```

>>> 8

Tool calling chaining example

```
"tool_calls": [  
  {  
    "id": "stock_call_1",  
    "function": {  
      "name": "get_stock_price",  
      "arguments": {  
        "ticker": "TSLA"  
      }  
    }  
  },  
  {  
    "id": "news_call_2",  
    "function": {  
      "name": "fetch_news_summary",  
      "arguments": {  
        "ticker": "TSLA"  
      }  
    }  
  }  
  [...]
```

High-level Concept

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the `Front Row media center` program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search `Front Row` next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search `Front Row (software)` .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: `Front Row (software)` is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish[keyboard function keys]`



ReAct loop

- Create a plan.
- Identify action using reasoning (CoT).
- Take action.
- Observe result and update plan.

Figure: <https://arxiv.org/abs/2210.03629>

Early examples

You are an agent that answers questions by using two actions:

- search[query]: Google search the query. You can also use it to calculate math.
- finish[answer]: return the answer

Your generation should have the following format:

Thought: to analyze the observation and inform the action.

Action: your action

Question: If I have seven trillion dollars today, can I buy Apple, Nvidia, and Microsoft? If not, how much more money do I need?

Figure:

<https://rdi.berkeley.edu/llm-agents/f24>

Thought: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.

Action: search["current market capitalization of Apple, Nvidia, and Microsoft"]

Observation: "Nvidia's market cap stood at \$1.832 trillion as of about 10 a.m. ET Wednesday, while Alphabet's was at \$1.814 trillion, bringing Nvidia behind just Microsoft (MSFT) at \$3.015 trillion and Apple (AAPL) at \$2.836 trillion."

Thought: Now that I have the market cap for all companies, let's add them together and see if \$7 trillion is enough.

Action: search["1.832 trillion + 3.015 trillion + 2.836 trillion"]

Observation: 7683000000000

Thought: The amount of money needed in addition to the \$7 trillion is \$683 billion.

Action: finish["You need an additional \$683 billion to be able to buy Apple, Nvidia, and Microsoft."] ✓

Figure:

<https://rdi.berkeley.edu/llm-agents/f24>

Failure Modes Persist

Android is at signing validation and integrity config now, very near bundle output.

🌐 ウェブを検索 ▼

weather: Seoul, South Korea

I made an irrelevant tool call by accident there; ignoring it and continuing the release work. Android is still the active task.

Gradle emitted the usual daemon/metaspaces warning but is continuing. It's in lint-vital analysis now.

Figure: <https://x.com/i/status/2072864322536222875>

Do not blindly trust the system

LLMs can still hallucinate, even when reasoning and using tools.

Section 2

Tool-Based Reasoning using ReAct

Subsection 2

Standardization

Agency levels

Agency Level	Description	Short name	Example Code
☆☆☆	LLM output has no impact on program flow	Simple processor	<code>process_llm_output(llm_response)</code>
☆☆☆	LLM output controls an if/else switch	Router	<code>if llm_decision(): path_a() else: path_b()</code>
★★☆	LLM output controls function execution	Tool call	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★☆	LLM output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
★★★	One agentic workflow can start another agentic workflow	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>
★★★	LLM acts in code, can define its own tools / start other agents	Code Agents	<code>def custom_tool(args): ...</code>

Figure: https://huggingface.co/docs/smolagents/en/conceptual_guides/intro_agents

MCP: What is it?

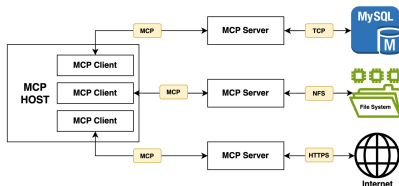


Figure: <https://velog.io>

Definition (MCP)

Model Context Protocol is a standardized framework that connects LLMs with external tools, APIs, and live data sources.

MCP: What is it?

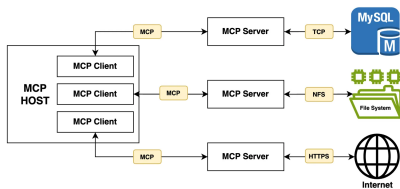


Figure: <https://velog.io>

Components:

- **MCP Server:** Backend executing standardized API calls.
- **MCP Client:** Translates LLM command to the MCP server.
- **MCP Host:** The LLM and UI.

MCP: Real-Use Example

what is a MCP server



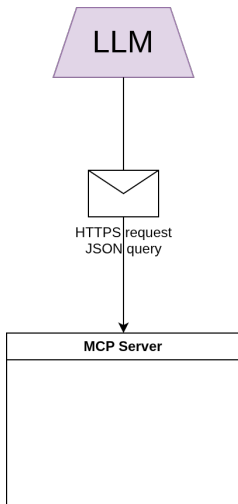
Thought

- **E. Key Benefits (Why it matters):** List the practical advantages (standardization, real-time data, scalability).
 - **F. Summary/Conclusion (TL;DR):** Reiterate the main point simply.
5. **Review and Refine (Tone Check):** Ensure the language is clear, professional, and accurately reflects the technical complexity while maintaining the accessibility provided by the source material. (Self-Correction: Make sure to mention that MCP is driven by Anthropic, as noted in both sources.)
6. **Final Output Generation.** (This leads to the detailed response below.)

Based on the provided documents, here is a comprehensive explanation of what an MCP server is.

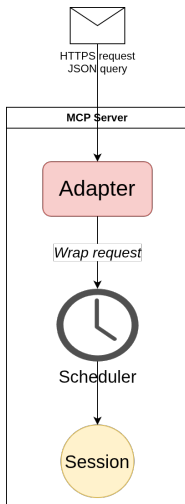
```
11:00:26-308421 INFO    Using search query: what is an MCP server
11:00:27-857375 ERROR    Error downloading https://medium.com/@ishosting/what-are-mcp-servers-and-how-do-they-work-74a9344be5cf: 403 C
11:00:28-539045 INFO    Added 2 successful web search results as attachments.
prompt processing progress, n_tokens = 2804, batch.n_tokens = 1020, progress = 0.998576
prompt eval time =      797.76 ms / 2808 tokens (    0.28 ms per token, 3519.86 tokens per second)
eval time =      32125.90 ms / 1907 tokens (   16.85 ms per token,   59.36 tokens per second)
total time =      32923.65 ms / 4715 tokens
11:01:01-611738 INFO    Output generated in 32.93 seconds (57.91 tokens/s, 1907 tokens, context 2808, seed 1700215113)
```

MCP: Internals



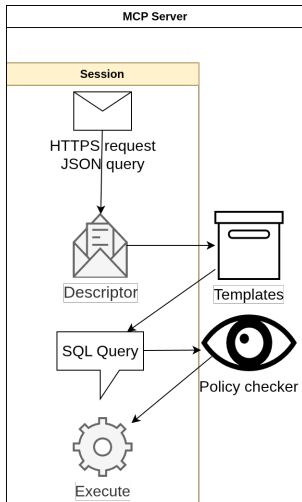
- The **MCP Client** sends a JSON query.
- Query wrapped in a HTTPS request for validation and security.

MCP: Internals



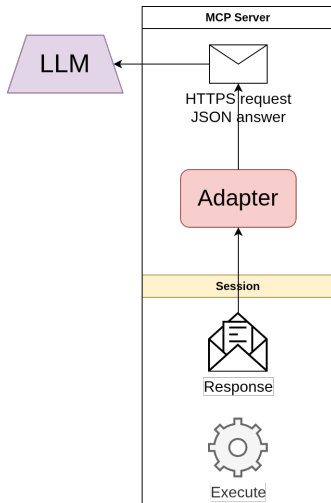
- The **Adapter** receives the request.
- Attaches metadata (id, length etc.).
- Forwards the message to the **Scheduler** which creates a session for the user.
- Enables concurrent requests by multiple users.

MCP: Internals



- Adapter's message contains a pointer to an **existing template** (e.g., `customer.balance`).
- Retrieve standardized template (e.g., `SELECT balance FROM Accounts WHERE ...`)
- Ensure that query has **necessary permissions** (in this case, `READ` access).

MCP: Internals



- Execute query with **limited permissions** and **tag** it (e.g., READ label) in case of further processing in the server.
- Adapter unpacks response and **re-packages** it into a JSON object inside an HTTPS response.

Summing up

ReACT

- The ReACT methodology was developed to allow autonomous use of tools by LLMs.
- This methodology combines a sandbox environment with standard and CoT prompting.
- Further standardization led to LLM agents with increasing autonomy as well as dedicated software architectures.
- The MCP architecture is the go-to modern solution for LLM tool use.

Outline

1 LLMs as System Components

- Inherent LLM limitations
- When to use LLMs?

2 Tool-Based Reasoning using ReAct

- Thought, Action, Observation
- Standardization

3 Agent-to-Agent Interaction

- Single-Agent vs. Multi-Agent
- Orchestrating Interaction
- What do we actually need?

Summing up

Agent-to-Agent Interaction

- Using simple prompting may not be enough for less well-defined tasks.
- Agent-prompting provides the models with a new frame of reference.
- Sometimes, it may be needed to use multiple frames of reference, necessitating multiple agents.
- When multiple agents are present, we need to establish a communication protocol to coordinate them.

Part II

Cost, Scalability, Accountability

Outline

4 Opex vs capex in LLM systems

- Local vs. Cloud Models
- Estimating Costs

5 Inference Optimization

- Quantization
- KV Caching

6 Beyond the Money Constraints

- GDPR
- EU AI Act
- OpSec

Section 4

Opex vs capex in LLM systems

Subsection 1

Local vs. Cloud Models

Different Cost Profiles

	OpEx	CapEx
Constraints	Volume	Capabilities
Funding	Continuous	Upfront
Consumption	Immediate	Depreciation
Local Model	Low	Variable
Propr. Model	High	None
Human	Very High	None

Table: <https://arxiv.org/abs/2503.16505>

There is no right or wrong answer generally.
Depending on your resources, either option is justifiable.

Local vs Proprietary: Costs

Local LLMs

- Need some setup.
- Need infrastructure.
- Very low inference costs.
- Can generally host smaller models.

Propr. LLMs

- No setup.
- No infrastructure.
- Medium-to-very high costs depending on provider.
- Very capable models, but extremely expensive.

The comparison doesn't end here! We will come back to this comparison later on in this lecture.

Analyzing Costs

Local LLMs

- Calculate hardware depreciation.
- Add electricity costs (depending on utilization).
- Add personnel and other costs.

Propr. LLMs

- Most companies bill by **input** and **output** tokens.
- Output tokens much more **expensive**.
- But input tokens much more **numerous** (context).
- Exact calculation varies by provider.

Section 4

Opex vs capex in LLM systems

Subsection 2

Estimating Costs

A Framework for Estimating Costs

- There is currently no agreed-upon framework for estimating costs for LLMs.
- But we can still try!
- Let CPT (Cost Per Task) the cost for completing an arbitrary task.
- Then, Total Cost = $\text{CPT} \times N_{\text{tasks}}$

Example: Product reviews

The task is to find in a single webpage how many times our product was mentioned. We crawl an average of 10,000 pages per day.

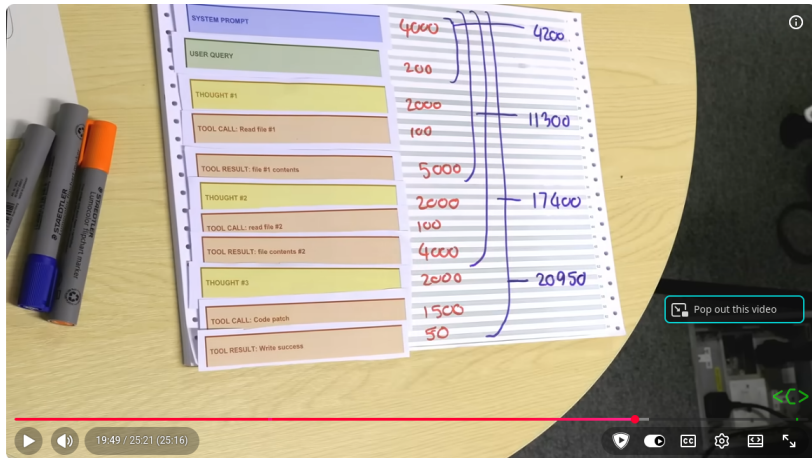
Local Hosting Costs

- $TCO = CapEx + PowerCost.$
- $CapEx = C_{\text{server}} \cdot N_{\text{servers}} \cdot \frac{D}{365 \cdot Y}$
- *PowerCost* can be estimated empirically.
- Keep in mind that *PowerCost* is partially a function of inference speed; the longer each task takes, the more the GPU is span up.

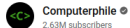
Reminder

The purpose of this estimation is not to have a solid budgetary limit, but rather for comparing available options.

Proprietary Model Costs



Why AI Tokens are so Expensive - Computerphile



Computerphile
2.63M subscribers

Subscribe

15K

114

Share

Save

...

Comparing Costs Between Options

Example: Social Science Experiments

	CPT	Total cost	Infrastructure
Humans	\$ 9.31	\$ 16,758	None
GPT-5.1	\$ 0.25	\$ 446.32	None
Local Host	\$ 0.03	\$ 58.82	GPU server
Small models	\$ 0.01	\$ 9.80	Consumer-grade

Table: <https://arxiv.org/abs/2503.16505>

Summing up

OpEx vs CapEx in LLM systems

- Choosing an LLM platform requires a trade-off between immediate, fixed costs and constant, rolling costs.
- Locally hosted LLMs require dedicated infrastructure, but are very cheap to run.
- Cloud-based LLMs require no infrastructure, but are much more expensive to run.
- There is no reliable formula for calculating costs, but an estimation with clear assumptions is invaluable for planning.

Outline

4 Opex vs capex in LLM systems

- Local vs. Cloud Models
- Estimating Costs

5 Inference Optimization

- Quantization
- KV Caching

6 Beyond the Money Constraints

- GDPR
- EU AI Act
- OpSec

Section 5

Inference Optimization

Subsection 1

Quantization

Reminder: What is Quantization?



Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

What is it?

- Quantization refers to dropping the size (not number) of the LLM parameters.
- Normally, each parameter is stored in a 32 bit floating point.
- Quantization can drop them to usually 4 bits, resulting in **×8 less** GPU memory and inference costs.

Reminder: What is Quantization?

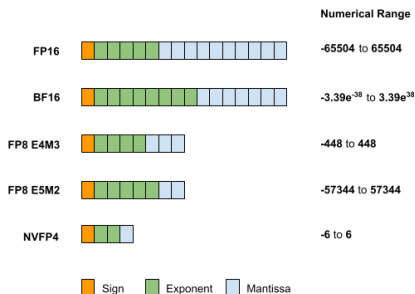


Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

Why is it useful?

- Quantization is one of the **most useful tools** at your disposal.
- Allows for training and inference in consumer-grade hardware.

Floating Point Representations

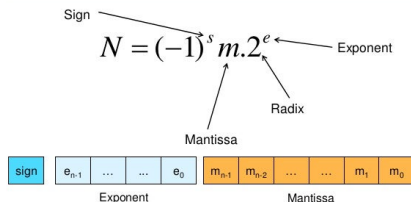


Figure: <http://www.slideshare.net/DilumBandara/07-data-representation>

- Real numbers are represented as “equations” in computers.
- 32 bits.
- **Exponent:** Determines the **range** of possible numbers.
- **Mantissa:** Determines **accuracy**.

FP quantization

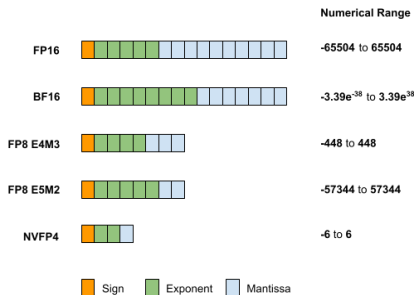


Figure: <http://www.slideshare.net/DilumBandara/07-data-representation>

- Works much like rounding numbers.
- Simple, intuitive, works well with GPUs.
- Many competitive techniques: AWQ, GGUF, GPTQ, ...

INT quantization

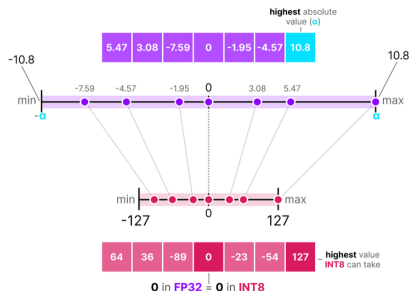


Figure: <https://www.maartengrootendorst.com/blog/quantization>

- Maps each float to an integer grid.
- Two kinds of errors: **rounding** error (float $\rightarrow$ int), and **clipping** error (int $\rightarrow$ grid range).
- Extremely efficient with CPUs and specialized hardware.

Which to Use?

Floating Point

- Specialized for GPUs.
- Well supported.
- Smaller errors.

Integer

- Extremely efficient in edge devices and specialized hardware.
- Very efficient for CPU inference.
- Not supported by GPUs.*
- Generally competitive with FP quantization.

Section 5

Inference Optimization

Subsection 2

KV Caching

What is it?

4

<bos>	x	-inf	-inf	-inf
Hello	x	x	-inf	-inf
World	x	x	x	-inf
!	x	x	x	x

4

<bos> Hello world !

Figure: <https://huggingface.co/blog/not-lain/kv-caching>

The problem:

- Transformers are autoregressive.
- The more they generate, the harder it gets to generate.
- Inference becomes burdened in long contexts.

What is it?

4

<bos>	x	-inf	-inf	-inf
Hello	x	x	-inf	-inf
World	x	x	x	-inf
!	x	x	x	x

4

<bos> Hello world !

However:

- KV matrices are same for all previous steps.
- We can simply store them instead of re-computing them.
- Thus; a cache for KV matrices!

Figure: <https://huggingface.co/blog/not-lain/kv-caching>

What is it?

		4				
<bos>	x	-inf	-inf	-inf		
Hello	x	x	-inf	-inf		
World	x	x	x	-inf	4	
!	x	x	x	x		
	<bos>	Hello	world	!		

Figure: <https://huggingface.co/blog/not-lain/kv-caching>

KV caching:

- Cache size stored in GPU.
- Scales linearly with context.
- $KV_cache(bytes) = batch_size \times context_length \times 2 \times num_layers \times num_kv_heads \times head_dim \times bytes_per_param$

What is it?

4

<bos>	x	-inf	-inf	-inf
Hello	x	x	-inf	-inf
World	x	x	x	-inf
!	x	x	x	x

4

<bos> Hello world !

Upsides:

- **Significant** speedup during inference.
- **Better** speedup as generation becomes **longer**!

Figure: <https://huggingface.co/blog/not-lain/kv-caching>

Downsides

LLaMa-8B (32K)

$$\begin{aligned} KV_cache &= 1 \times 32,768 \times \\ &2 \times 32 \times 8 \times 128 \times 2 = \\ &4,294,967,296 \text{ bytes} = \\ &4.0 \text{ GB} \end{aligned}$$

LLaMa-8B (128K)

$$\begin{aligned} KV_cache &= 1 \times 131,072 \times \\ &2 \times 32 \times 8 \times 128 \times 2 = \\ &17,179,869,184 \text{ bytes} = \\ &16.0 \text{ GB} \end{aligned}$$

Especially problematic for quantized models

LLaMa-8B 4-bit quantization: $\approx 5 \text{ GB}$.

Scaling Caches

Still in experimental stages.

- Sliding window approach.
- Context pruning.
- Quantization.

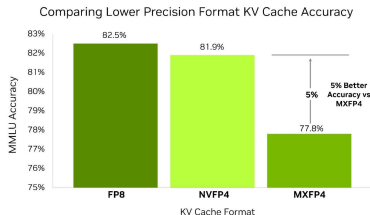


Figure: <https://developer.nvidia.com/blog/optimizing-inference-for-long-context-and-large-batch-sizes-with-nvfp4-kv-cache>

Scaling Caches

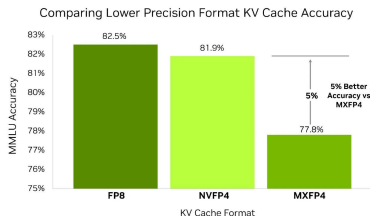


Figure: <https://developer.nvidia.com/blog/optimizing-inference-for-long-context-and-large-batch-sizes-with-nvfp4-kv-cache>

Still in experimental stages.

- Sliding window approach.
- Context pruning.
- Quantization.

But...

Perhaps, you don't need all that context in the first place!

Summing up

Inference Optimization

Two main ways of reducing runtime costs:

- 1 **Quantization** makes the model smaller. The level and type of quantization is a trade-off between capabilities and costs. Integer quantization is the most efficient type, but requires specialized hardware.
- 2 **KV-caching** is model-agnostic and can be deployed at no initial cost. It trades GPU VRAM for GPU compute. Always recommended to some degree, although the size should be tuned to the needs of the application.

Outline

4 Opex vs capex in LLM systems

- Local vs. Cloud Models
- Estimating Costs

5 Inference Optimization

- Quantization
- KV Caching

6 Beyond the Money Constraints

- GDPR
- EU AI Act
- OpSec

Section 6

Beyond the Money Constraints

Subsection 1

GDPR

GDPR



General Data Protection Regulation

- Privacy law concerning almost exclusively **personal data**.
- Any information relating to an identified or identifiable natural person (“data subject”).

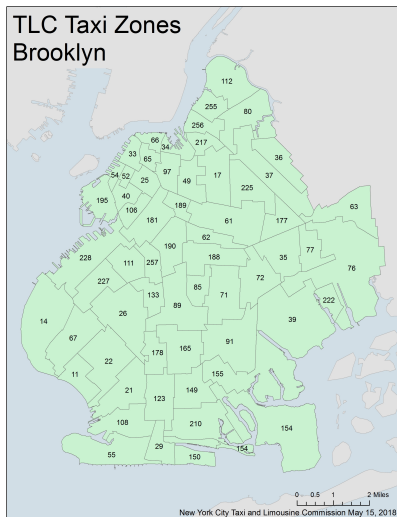
GDPR



Where does it apply? If any are true:

- Data subjects are in the Union.
- Their behavior takes place in the Union.
- Controller or processor in the Union.

Identifiable Information



Direct Identification

Including name, address
etc.

Indirect Identification

- Combination of SDB characteristics.
- ID number, location.
- Hardware/IP address, browser fingerprinting.

Frontline roles

Definition (Data Controller)

The natural or legal person, public authority, agency or other body which, alone or jointly with others, **determines the purposes and means** of the processing of personal information.

Definition (Data Processor)

A natural or legal person, public authority, agency or other body which **processes** personal data **on behalf of the controller**.

Frontline roles

Definition (Data Controller)

The natural or legal person, public authority, agency or other body which, alone or jointly with others, **determines the purposes and means** of the processing of personal information.

Definition (Data Processor)

A natural or legal person, public authority, agency or other body which **processes** personal data **on behalf of the controller**.

That's you!

Section 6

Beyond the Money Constraints

Subsection 1

GDPR

How Should I Comply?

The Seven Principles

- 1 **Lawfulness, Fairness, and Transparency**
- 2 **Purpose Limitation:** Data must be collected for specified, explicit, and legitimate purposes.
- 3 **Data Minimisation/Proportionality:** Data collected must be adequate, relevant, and strictly limited to what is necessary for the declared aim.
- 4 **Accuracy:** Personal data must be accurate and, where necessary, kept up to date.

How Should I Comply?

The Seven Principles

- 5 **Storage Limitation:** Data must only be kept in a form that permits identification for no longer than is necessary for the processing purpose.
- 6 **Integrity and Confidentiality (Security):** Data must be protected by appropriate technical and organizational measures against unauthorized access, loss, or damage.
- 7 **Accountability:** Controllers must actively implement and continuously demonstrate compliance with all data protection provisions in their activities.

How Should I Comply?

Data protection by design/by default

- Design your data collection and analysis **proactively**.
- Maximum privacy is the default. Users can opt out.
- Keep users informed. Keep the system user-centric.
- Keeping **minimal personal data** $\Leftrightarrow$ minimum risk and complications.

Data Protection Officer (DPO)

Data Controllers are typically required to have a **designated expert** trained on this legislation.

How Far Should I Go?

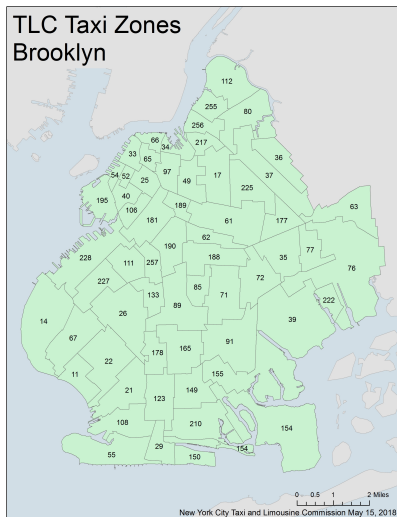
High-risk systems

- Evaluation or scoring, including profiling and predicting
- Automated-decision making with legal or similar significant effect
- Datasets that have been matched or combined
- Data concerning vulnerable data subjects

How do I Remove Personal Information?

- Collect data without personal information (Data minimization principle).
- Anonymize: All identifying elements are eliminated from a set of personal data so that the data subject is no longer identifiable.
- Redaction/Expungement: “Right to be forgotten”.

How do I Remove Personal Information?



Pseudo-anonymization is not enough

Attributes, such as name, date of birth, sex, address, or other elements that could lead to identification are replaced by a pseudonym.

Summing up

GDPR

- Privacy regulation in the EU but not limited inside its borders.
- Places constraints on what data you, as Data Processors, can store and use.
- Generally avoid collecting any personal data.
- If you can not, anonymize them.
- If you can not, follow established principles and procedures. Talk to your DPO.

Section 6

Beyond the Money Constraints

Subsection 2

EU AI Act

EU AIA



- Supplementary to other legislation (e.g., GDPR).
- Applicable to all AI systems in the Union.
- Classify systems by internal risk, restrict them appropriately.

Scope



Where does it apply?

- **Providers** placing AI systems in the Union.
- **Users** of AI systems in the Union.
- Importers, distributors, manufacturers, representatives and affected persons using or operating systems whose **outputs** are used within the Union.

Scope



Where does it NOT apply?

- National security & defense.
- Research / RnD (excluding live testing).
- Purely personal activities.

What is an AI System?

Definition (AI System)

- A **machine**-based system.
- Operate with varying levels of **autonomy**.
- **Infers** how to generate outputs from input.
- (Optional) Adaptive after deployment

Section 6

Beyond the Money Constraints

Subsection 2

EU AI Act

Restrictions

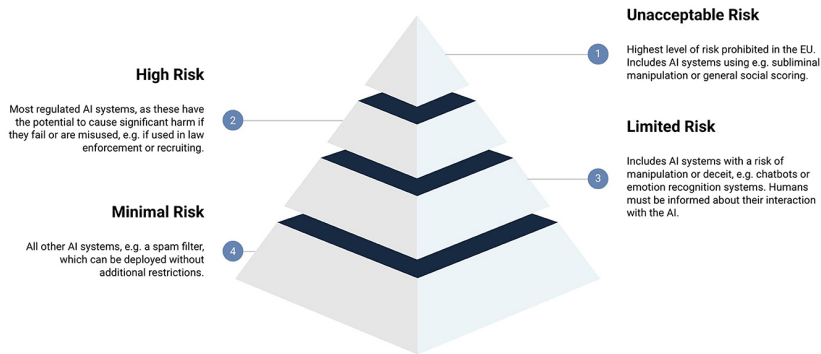


Figure: Image by <https://www.frontiersin.org/journals/political-science/articles/10.3389/fpos.2024.1451601/full>, released under CC.

Restrictions: Unacceptable Risk

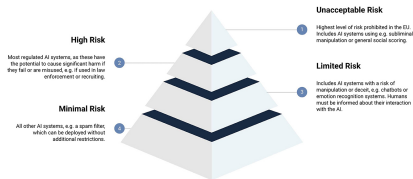


Figure: Image by <https://www.frontiersin.org/journals/political-science/articles/10.3389/fpos.2024.1451601/full>, released under CC.

Unacceptable Risk

- System directly violates core EU principals.
- **Prohibited.**
- Example: Emotion recognition in the workplace, unless for medical or safety reasons;

Restrictions: Unacceptable Risk

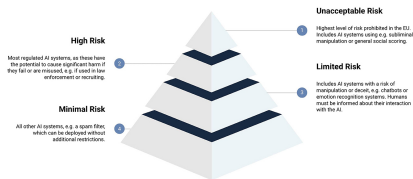


Figure: Image by <https://www.frontiersin.org/journals/political-science/articles/10.3389/fpos.2024.1451601/full>, released under CC.

High Risk

- Failure may endanger health and safety.
- Examples: Critical infrastructure, hiring, essential services.
- Requires development of a **risk management system**.

Restrictions: High Risk

A high-risk system must include:

- A Risk Management System.
- Data Governance.
- Technical documentation.
- Logging system.
- Transparency for human users.
- Human oversight.
- Minimum information use.
- Ensure accuracy, robustness.

Summing up

EU AIA

- Supplementary regulation to GDPR, only applies to AI systems.
- Unlike GDPR, you can not avoid it.
- Ensure your system is not classified as Unacceptable Risk.
- If your system could possibly be classified as High Risk, administrative, technical and procedural measures must be implemented.

Section 6

Beyond the Money Constraints

Subsection 3

OpSec

What is it?

Definition (OpSec)

NIST SP 800-53: A process by which potential adversaries can be denied information about capabilities and intentions[...].

What is it?

Definition (OpSec)

NIST SP 800-53: A process by which potential adversaries can be denied information about capabilities and intentions[...].

Information that your institution wants to keep secret.

Why do we care?

If you are working with:

- Personal data.
- Medical or otherwise sensitive data.
- Internal data (such as customer details).

Then it is **crucial** that these data remain in your institution.

Applies both if:

- Your system uses LLM agents.
- You yourself use LLM agents.

Proprietary models



- Many providers explicitly state that input data can be used for further model pretraining.
- There is no guarantee that API-based models respect user agreements on safeguarding input.
- See: Benchmark leakage.

Solutions

No agreed-upon strategy.

- Identify attack surface. Which data do we need to keep private?
- Anonymization.
- Use locally hosted models for sensitive tasks.

Part III

System Evaluation

Outline

7 General Methodologies

- Why Evaluate?
- Objectives, Criteria and Metrics
- Implementation

8 Use Cases

- Benchmarking
- LLM Simulations

Section 7

General Methodologies

Subsection 1

Why Evaluate?

Reasons for evaluation

LLMs:

- Are unpredictable.
- LLMs make mistakes, even in tasks that seem easy to us.
- Unlike rule-based and traditional ML systems, they can give us **any kind** of output.

Naive approaches

Some commonly used approaches:

- Use standardized benchmarks.
- Use extremely large models.
- Run believability studies.
- Do not run evaluation.

Unexpected failures

Not very	False	False	5
refer not to say	True	False	5
Not very	False	True	1
No	False	True	5
Somewhat	False	True	1
No	True	False	I'm sorry
No	True	True	1
Somewhat	True	True	3

Example: Annotation

- Output: set of integers.
- LLM output could be any kind of string.
- Safety tuning may prohibit the model from running the task.

Section 7

General Methodologies

Subsection 2

Objectives, Criteria and Metrics

Objectives

The first step towards evaluating a system is identifying what it is trying to achieve.

- Is it used as a chatbot-assistant? For internal or external use?
- Is it used as a step in a processing pipeline?
- Is it completely automated? Does it interface with human operators?
- Is it used for simulations?

Example: LLM for internal documentation

Objective: Give guidance to team members for internal information.

Criteria

For each of your objectives find a set of criteria which measure whether that objective is satisfied.

Example: LLM for internal documentation

Criteria:

- Does the LLM demonstrate knowledge of facts private to the organization?
- Are outputs updated when new knowledge is inserted?
- Is sensitive information only given to authorized users?

Metrics

For each of your criteria find a set of metrics to measure whether the criteria are satisfied.

Example: LLM for internal documentation

Criterion: Does the LLM demonstrate knowledge of facts private to the organization?

Metrics:

- F1 for retrieved documents.
- % test queries whose answers contain non-public knowledge.

Section 7

General Methodologies

Subsection 3

Implementation

Summing up

LLMs as system components

- Ordinary tasks that are trivial to humans or through programming may be exceptionally hard for LLMs.
- This fact motivates us to combine them with standard tools instead of relying on them for everything.
- Generally, LLMs should be thought of as functions which can adapt to the input.

How do we implement testing?

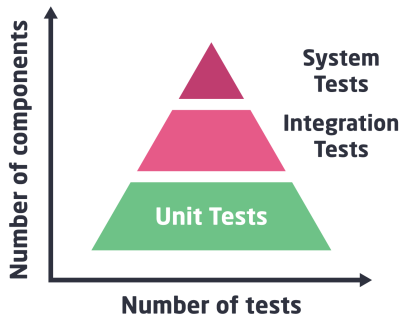


Figure:

<https://symflower.com/en/company/blog/2023/unit-testing-vs-integration-testing/>

If LLMs can be thought of as functions, they can be tested as such. We can verify:

- 1 Basic input/output using **unit** testing.
- 2 Interactions with other components through **integration** testing.
- 3 System behavior and environmental interaction through **system** testing.

How do we implement testing?

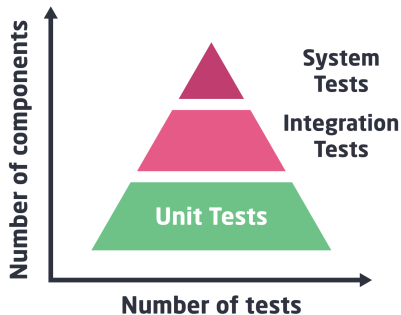


Figure:

<https://symflower.com/en/company/blog/2023/unit-testing-vs-integration-testing/>

Unit Testing

- Give sample instructions to the LLM and inspect whether output is well-formed.
- Test what happens under faulty/no output.
- Provide some questions whose answer is unambiguous.

How do we implement testing?

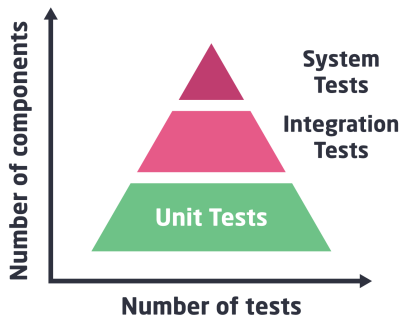


Figure:

<https://symflower.com/en/company/blog/2023/unit-testing-vs-integration-testing/>

Integration Testing

Think of the LLM as an API endpoint.

- What happens if the LLM doesn't respond quickly enough?
- Or when it doesn't respond at all?
- Create adapter classes for testing components interacting with the LLM.

How do we implement testing?

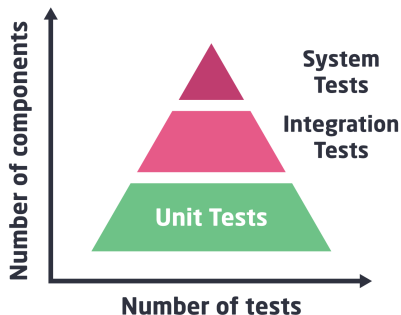


Figure:

<https://symflower.com/en/company/blog/2023/unit-testing-vs-integration-testing/>

System Testing

- Execute the whole system.
- If completely automated, run the pipeline with realistic data.
- Observe whether the system functions correctly w.r.t. results, speed, cost etc.

How do we implement testing?

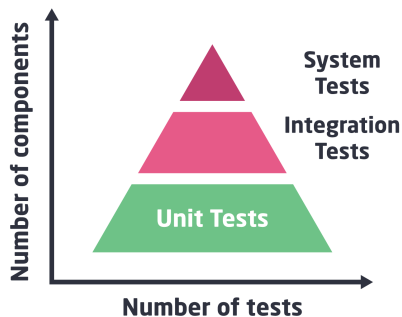


Figure:

<https://symflower.com/en/company/blog/2023/unit-testing-vs-integration-testing/>

- Testing should be performed continuously, before each deployment and during development.
- Due to LLM's computational constraints, some tests may not be able to run continuously.
- It may be necessary to swap to smaller models for speed and cost efficiency.

HCI Evaluation

- If your system includes any kind of human interaction, it is advisable to also use Human Computer Interaction (HCI) testing.
- These tests should cover both the front-end aspects of your system (UI/command line) and the model's overall interaction with the user.

HCI Evaluation

Definition (HCI)

- HCI Evaluation tests the quality of the interface between human and system.
- It ensures the technology is not just functional, but also usable and satisfying.

Goals

- Assess system functionality and efficiency.
- Identify specific design problems (pain points).
- Measure overall user experience and satisfaction.

HCI Evaluation

Expert-Based Reviews:

- **Heuristic Evaluation:** Experts check the UI against established usability principles. (Fast, cheap)
- **Cognitive Walkthrough:** Experts simulate user learning paths for specific tasks.
- **Upside:** Ideal for early-stage design; quick detection of systemic flaws.
- **Drawback:** Lacks real-world context; findings are subjective.

HCI Evaluation

User-Based Testing:

- Usability Testing: Users perform tasks while verbalizing their thoughts (Think-Aloud).
- SUS Score: A standardized questionnaire that measures perceived overall usability.
- **Upside:** Provides rich, realistic data on actual user friction.
- **Drawback:** Can be costly; results may be influenced by conscious user behavior.

HCI Evaluation

Contextual Insight:

- **Field Studies:** Observing users in their actual environment (real context).
- **Interviews:** Deep, one-on-one discussions to uncover motivations and unmet needs.
- **Upside:** Provides deep, rich insight into the full workflow (ecological validity).
- **Drawback:** Very time-intensive; difficult to generalize findings broadly.

HCI Evaluation

Choosing the Right Strategy

- **Formative Evaluation:** During design, use experts and qualitative methods.
- **Summative Evaluation:** After design, use quantitative and user testing.
- **Decision Factors:** Consider time, resources, desired objectivity, and evaluation stage.
- **Summary:** Combining methods ensures a holistic and comprehensive assessment.

Summing up

General Evaluation Methodologies

- Evaluation is needed, since LLMs are by their nature volatile and unpredictable.
- We first need to find the objectives of our system, then criteria to judge these objectives, and metrics to measure the adherence to our criteria.
- We implement the metrics using unit, integration and system tests. If humans are involved in the system, we should also consider HCI testing.

Outline

7 General Methodologies

- Why Evaluate?
- Objectives, Criteria and Metrics
- Implementation

8 Use Cases

- Benchmarking
- LLM Simulations

Section 8

Use Cases

Subsection 1

Benchmarking

A Note on Benchmarks

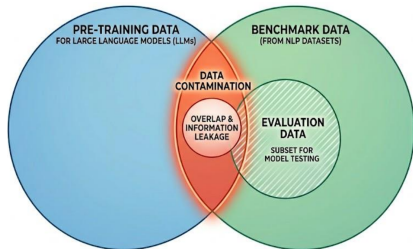


Figure: Credit: Electra Maria Papadopoulos.

Data Contamination

- Evaluation data appears in training.
- Web-scraping may include datasets used as training, validation, or benchmarks, unknowingly inflating reported performance.
- Sometimes on purpose (“benchmaxxing”).

A Note on Benchmarks

Definition (Benchmaxxing)

Optimizing a LLM's performance primarily for high scores on public **leaderboards** and **benchmarks**, rather than practical, and reliable real-world capability.

How?

- Training Data Contamination.
- Format Overfitting.
- Checkpoint Cherry-Picking.
- Prompt Engineering.

What Are Benchmarks For?

- Use benchmarks primarily for **standardized, relative comparison**.
- Ask: *“Is this system better on the capability being measured?”*
- Treat scores as **signals**, not comprehensive measures of capability.
- Check whether the benchmark reflects the intended users, tasks, and context.

Look Beyond the Score

- Do not equate benchmark success with real-world usefulness.
- Examine what the metric **omits**: cost, safety, robustness, quality, workflow integration.
- Prefer **multiple complementary evaluations** over a single score.
- Use domain expertise and human evaluation where automated metrics are insufficient.

Use Cases

Research

- Purpose: **measure research progress.**
- Enable comparison across methods and studies.
- Interpret results within the benchmark's scope.

Deployment

- Purpose: **screen candidates.**
- Start with public benchmarks, then test domain-specific **actual use cases.**
- Use continuous evaluation where appropriate.

Section 8

Use Cases

Subsection 2

LLM Simulations

Why use LLMs?

- LLMs offer scalable, low-cost solutions for generating large volumes of simulated data.
- They handle complex instructions and subjective decision-making naturally.
- This overcomes traditional limitations of human subject data collection (cost, time, scarcity).

Why use LLMs?

Examples

- We want to test how users will click through a website.
- We want to test an advertisement targeting specific demographics.
- We research how human users react to a specific post in online fora.

LLM vs. Traditional ABM

- Classic ABM (Agent-Based Modeling) relies on pre-defined, simple interaction rules.
- LLM agents surpass this by enabling flexible, natural-language-based behavior modeling.
- LLMs allow for richer simulation of subjective human thought and diverse reasoning.

Inherent LLM Limitations

- LLMs tend toward generating “average personas”.
- Their training maximizes high-frequency responses, limiting the capture of rare behaviors.
- LLMs inherit and amplify social and demographic biases from training data.
- They struggle to accurately model minority groups or specific cultural subgroups.
- Using biased data risks reinforcing existing inequalities and misrepresenting populations.

Foundational Validation

- **Data Analytics:** Essential for ensuring input/output data integrity and relevance.
- **Empirical Validation:** Compares simulation output against real-world observations.
- **Sampling & Visualization:** Used to explore the simulation space and visualize emergent patterns.
- **Docking:** Aligns GABM outputs with those of other state-of-the-art models.

These methods establish baseline structural soundness and predictive capacity.

Microface Validation

- Checks if individual agent actions and properties align with human reality.
- For GABMs, this requires two distinct validations:
 - Assessing the coherency of decision-making processes.
 - Validating the realism and consistency of natural language interactions.

Macroface Validation

- Assesses if the emergent system-wide patterns correspond to real-world phenomena.
- Techniques include Bootstrapping, Causal Analysis.
- This is necessary for systems with thousands of agents where individual checks are impractical.

Face Validation

- **Animation Assessment:** Experts review simulation replays for dynamic realism.
- **Output Assessment:** Human experts check the plausibility and coherence of the simulation outputs.
- **Immersive Assessment:** Validators impersonate agents to directly evaluate believability and reaction complexity.